

Rust & esp-rs 전용 Jenkins CI/CD

타깃 GitHub 저장소 자동 배포 파이프라인

기설치 환경(rust, esp-rs, sccache, gh auth)을 100% 활용하는 초고속 임베디드 빌드 아키텍처

Node Stack: Native Rust, esp-rs (espup), sccache, gh CLI (Pre-authenticated)

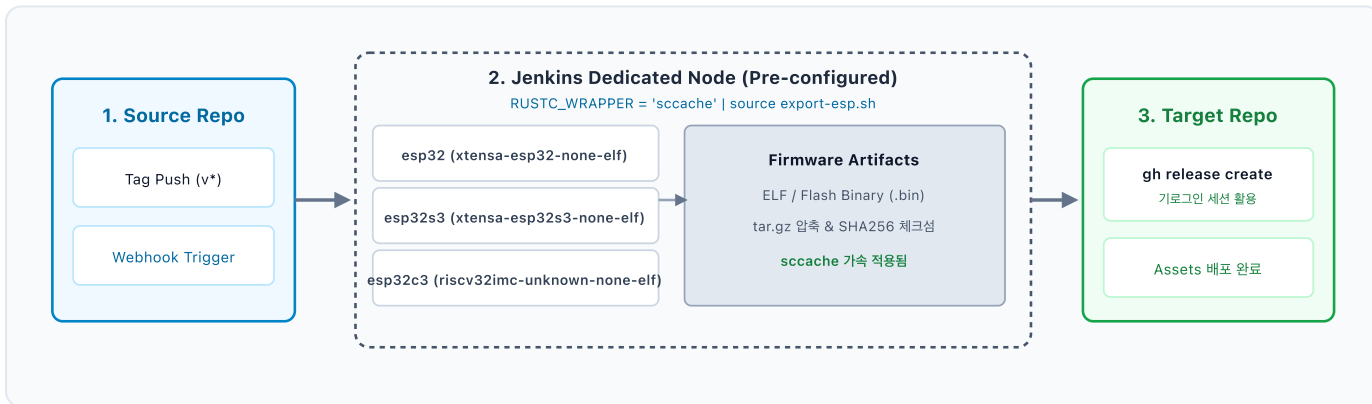
Target Artifacts: Multi-target Firmware Binaries (.bin / .elf)

1. 변경된 노드 환경 분석 및 파이프라인 최적화

빌드 노드에 `rust`, `esp-rs`, `sccache`, `gh` (로그인 완료) 가 이미 준비되어 있으므로, 기존의 무거운 Docker 기반 `cross` 컴파일 및 Jenkins Credential 주입 로직이 대폭 간소화되고 빌드 속도가 극대화됩니다.

⚡ 기설치 환경 기반 4대 최적화 포인트

- cross (Docker) 제거** → 네이티브 **cargo** 빌드: 노드의 **esp-rs** 툴체인(Xtensa / RISC-V)을 직접 호출하므로 Docker 컨테이너 오버헤드가 없습니다.
- sccache 기본 연동**: `RUSTC_WRAPPER = 'sccache'` 환경변수를 통해 동일 코드/종속성 재빌드 시간을 70~80% 단축합니다.
- Jenkins Credential 생략**: 노드에 `gh auth login` 이 완료되어 있으므로 별도 시크릿 토큰 주입 없이 `gh release create / upload` 명령이 바로 수행됩니다.
- export-esp.sh 자동 로드**: espup으로 구축된 Xtensa/LLVM 환경변수를 파이프라인 셀에서 자동으로 로드하여 임베디드 툴체인 경로 문제를 해결합니다.



2. 최적화된 Jenkinsfile (Declarative Pipeline)

노드 환경에 맞춰 작성된 최종 파이프라인 스크립트입니다. ESP 타겟 칩셋(`esp32` , `esp32s3` , `esp32c3`)을 병렬로 빌드하고, `sccache` 통계를 출력하며, 로그인된 `gh` 로 릴리스를 생성합니다.

```

pipeline {
    agent any

    options {
        timeout(time: 1, unit: 'HOURS')
        disableConcurrentBuilds()
        ansiColor('xterm')
    }

    environment {
        // 1. 배포 대상 GitHub 저장소 (owner/repository)
        TARGET_REPO = 'your-org/target-binary-repo'

        // 2. sccache 컴파일 캐시 활성화
        RUSTC_WRAPPER = 'sccache'
        SCCACHE_DIR = "${WORKSPACE}/.sccache"

        // 3. 터미널 출력 컬러 유지
        CARGO_TERM_COLOR = 'always'
    }

    stages {
        stage('Checkout & Versioning') {
            steps {
                checkout scm
                script {
                    // Git 태그 또는 최신 커밋 해시 추출
                    env.RELEASE_TAG = sh(
                        script: 'git describe --tags --always',
                        returnStdout: true
                    ).trim()
                    echo "=== Target Release Version: ${env.RELEASE_TAG} ==="
                }
            }
        }

        stage('Pre-flight Check') {
            steps {
                sh '''
                    echo "=== Toolchain Verification ==="
                    cargo --version
                    rustc --version
                    sccache --version

                    # gh CLI 로그인 세션 확인 (이미 로그인된 상태 검증)
                    gh auth status

                    # sccache 통계 초기화
                    sccache --zero-stats || true

                    mkdir -p dist
                    ...
                '''
            }
        }

        stage('Parallel esp-rs Build') {
            matrix {
                axes {
                    axis {
                        name 'CHIP'
                    }
                }
            }
        }
    }
}

```

```

        values(
            'esp32',
            'esp32s3',
            'esp32c3'
        )
    }
}

stages {
    stage('Build & Package') {
        steps {
            script {
                // 칩셋별 Rust Target Triple 매핑
                def targetTriple = ''
                if (CHIP == 'esp32') {
                    targetTriple = 'xtensa-esp32-none-elf'
                } else if (CHIP == 'esp32s3') {
                    targetTriple = 'xtensa-esp32s3-none-elf'
                } else if (CHIP == 'esp32c3') {
                    targetTriple = 'riscv32imc-unknown-none-elf'
                }

                def binName = 'my-firmware' // Cargo.toml 바이너리 이름
                def artifactName = "${binName}-${RELEASE_TAG}-${CHIP}.tar.gz"

                sh """
                    # 1. esp-rs 툴체인 환경 로드 (esput 기본 설정)
                    if [ -f "$HOME/export-esp.sh" ]; then
                        source "$HOME/export-esp.sh"
                    fi

                    # 2. 펌웨어 릴리스 빌드 (sccache 자동 적용)
                    echo "Building for \${CHIP} (\${targetTriple})..."
                    cargo build --release --target \${targetTriple}

                    # 3. 플래시용 바이너리 변환 (cargo-esptool 설치 시)
                    mkdir -p pkg_\${CHIP}
                    if command -v cargo-esptool > /dev/null 2>&1; then
                        cargo esptool save-image --release --chip \${CHIP} pkg_
\${CHIP}/\${binName}.bin

                    fi

                    # ELF 바이너리도 함께 아카이빙 (디버깅 / gdb용)
                    cp target/\${targetTriple}/release/\${binName} pkg_\${CHIP}/
\${binName}.elf

                    # 4. 아카이브 압축 (tar.gz)
                    cd pkg_\${CHIP}
                    tar -czvf ../dist/\${artifactName} *
                    cd ..

                    # 5. 무결성 검증 SHA256 체크섬 생성
                    cd dist
                    sha256sum \${artifactName} > \${artifactName}.sha256
                """
            }
        }
    }
}

```

```

stage('Publish to Target GitHub Repo') {
    steps {
        sh '''
            echo "Deploying release assets to https://github.com/${TARGET_REPO}..."

            # sccache 컴파일 캐시 히트율 확인
            echo "=== sccache Statistics ==="
            sccache --show-stats || true

            # 타깃 저장소에 릴리스가 이미 존재하는지 체크 (gh 사전 로그인 세션 활용)
            if gh release view "${RELEASE_TAG}" --repo "${TARGET_REPO}" > /dev/null 2>&

1; then
                echo "Release ${RELEASE_TAG} exists. Uploading new assets..."
                gh release upload "${RELEASE_TAG}" dist/* \
                    --repo "${TARGET_REPO}" \
                    --clobber
            else
                echo "Creating new release ${RELEASE_TAG} on ${TARGET_REPO}..."
                gh release create "${RELEASE_TAG}" dist/* \
                    --repo "${TARGET_REPO}" \
                    --title "Firmware Release ${RELEASE_TAG}" \
                    --notes "Automated esp-rs build from Jenkins Node (sccache accelerat
ed)."
            fi
        '''
    }
}

post {
    always {
        // 디스크 공간 절약을 위한 타깃 디렉터리 정리 (sccache는 보존됨)
        cleanWs(deleteDirs: true, notFailBuild: true, patterns: [
            [pattern: 'target/**', type: 'INCLUDE'],
            [pattern: 'pkg_*/**', type: 'INCLUDE']
        ])
    }
    failure {
        echo "Pipeline build failed. Please check console logs."
    }
}
}

```

3. 주요 변경 상세 해설

1) gh 사전 로그인 세션 활용

- 노드에서 이미 `gh auth login` 을 마쳤기 때문에 Jenkinsfile에 `GH_TOKEN = credentials(...)` 를 명시하지 않아도 `gh` 명령어가 노드의 `~/.config/gh/hosts.yml` 인증 설정을 자동 로드합니다.
- 타깃 저장소에 대한 쓰기 권한이 해당 gh 계정에 부여되어 있으면 `--repo your-org/target-binary-repo` 옵션만으로 모든 릴리스 업로드가 처리됩니다.

2) sccache 를 통한 컴파일 가속

- `RUSTC_WRAPPER = 'sccache'` 를 환경변수로 선언하여 `cargo build` 가 내부적으로 `sccache` 를 경유하도록 설정했습니다.
- 파이프라인 시작 시 `sccache --zero-stats` , 종료 시 `sccache --show-stats` 를 통해 빌드 캐시 히트율(Cache Hits)과 컴파일 절감 시간을 콘솔 로그에서 직접 모니터링할 수 있습니다.

3) esp-rs 전용 환경 로드 및 산출물

- `source "$HOME/export-esp.sh"` : espup 툴체인(Xtensa Clang/LLVM 등)은 셸에 환경변수가 로드되어 있어야 작동합니다. 각 빌드 스텝 앞단에서 이를 자동으로 소싱합니다.
- 임베디드 타깃 산출물은 디버깅용 `.elf` 와 장치 플래싱용 `.bin` (cargo-espflash)을 함께 패키징하여 배포용 `tar.gz` 와 `.sha256` 체크섬을 생성하도록 구성했습니다.